Edge/Fog/Cloud Task Placement Simulator

ARM 305 – Cloud, Dev, Edge and Fog Computing | Mini Project Report

(Team: Yash Kumar-01919051624,Yug Katyal-03819051624)

1. Objective

Modern applications mix latency-critical work (sensor processing, real-time control) with heavy, delay-tolerant work (analytics, batch jobs). Edge nodes are close to users but small, Fog nodes are mid-sized, and Cloud nodes are large but far away. Deciding where each task should run is the placement problem.

The objective of this project is to build a locally runnable Python simulator that:

- accepts computing tasks with CPU, memory and latency-sensitivity requirements;
- places each task on an Edge, Fog or Cloud node using rule-based algorithms;
- simulates resource use, virtual machine (VM) records and multi-tenant access control;
- reports placement decisions, simulated latency, utilization, and successful/failed task counts;
- compares two placement policies and runs overload, security and capacity/latency-change experiments.

No real cloud account, hypervisor or Docker is used. Everything is simulated in Python with pandas and matplotlib.

2. System Architecture

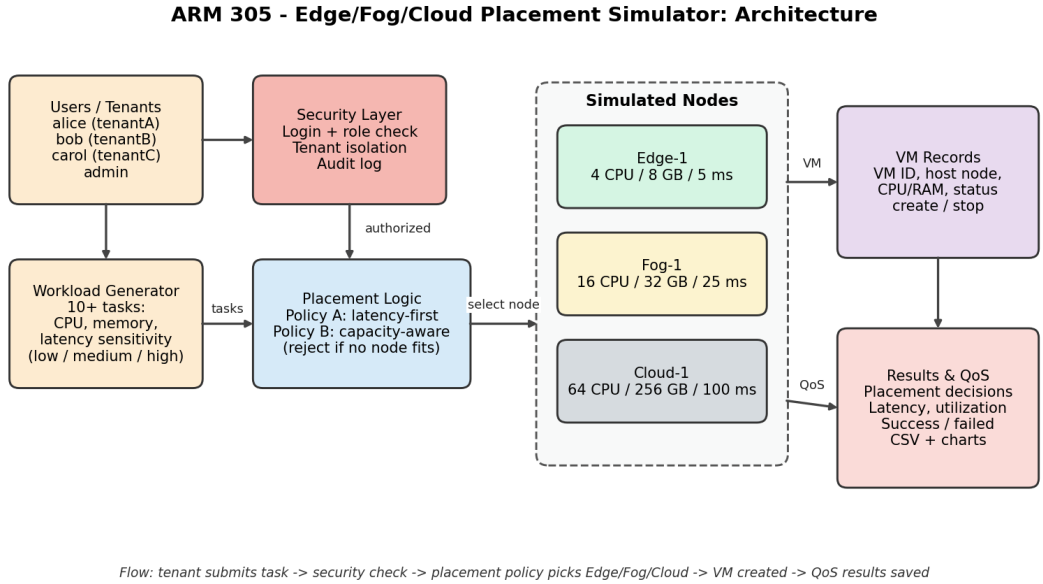


Figure 1: Architecture of the simulator.

The system has six parts:

1. **Users / Tenants** – demo users alice (tenantA), bob (tenantB), carol (tenantC) and admin. No real passwords are stored.
2. **Workload generator** – creates 10 or more tasks, each with a task ID, tenant, CPU, memory and latency sensitivity (low, medium, high). A fixed random seed makes runs reproducible.

3. **Security layer** – a role and tenant check that decides who may view a task record or stop a VM, plus an audit log.
4. **Placement logic** – two policies (latency-first and capacity-aware) that choose a node or reject the task.
5. **Simulated nodes** – one Edge, one Fog and one Cloud node, each with CPU, memory, base latency and current utilization.
6. **VM records and results** – every placed task creates a VM record; QoS results are saved to CSV and charts.

2.1 Node configuration

Node	Tier	CPU (vCPU)	Memory (GB)	Base latency (ms)
Edge-1	Edge	4	8	5
Fog-1	Fog	16	32	25
Cloud-1	Cloud	64	256	100

3. Placement Algorithms

3.1 Task model and SLA limits

Each task has a latency-sensitivity class. Each class has a latency limit (its SLA) that the simulated latency must not exceed:

Sensitivity	SLA limit
high	20 ms
medium	50 ms
low	200 ms

3.2 Utilization and latency model

Utilization of a node is the larger of its CPU and memory usage fractions:

```
utilization = max(used_cpu / total_cpu, used_ram / total_ram)
```

The simulated latency grows as a node gets busier:

```
simulated_latency = base_latency × (1 + utilization after placing the task)
```

For example, Edge-1 (5 ms) at full load gives 10 ms, Fog-1 (25 ms) at full load gives 50 ms, and Cloud-1 (100 ms) at full load gives 200 ms.

3.3 Policy A – Lowest-latency-first

1. Sort all nodes by base latency (Edge, then Fog, then Cloud).
2. Choose the first node that has enough free CPU and free memory for the task.
3. If no node fits, reject the task.

This policy ignores the task's SLA when choosing. It is simple, but it fills Edge first regardless of which tasks actually need it, so later latency-critical tasks spill to slower nodes.

3.4 Policy B – Capacity-aware

1. Keep only nodes with enough free CPU and free memory.
2. From those, keep only nodes whose estimated latency after placement is within the task's SLA limit.
3. Among the remaining nodes, choose the one with the lowest utilization after placement (ties broken by lower latency).
4. If no node qualifies, reject the task.

This policy matches tasks to nodes by both capacity and latency, spreads load, and keeps Edge free for latency-critical work. Its trade-off is that it rejects a task rather than place it where it would break its SLA.

3.5 Task outcomes

- **SUCCESS** – the task was placed and its simulated latency is within its SLA.
- **SLA_VIOLATED** – the task was placed, but its simulated latency exceeds its SLA (counted as failed).
- **REJECTED** – no node could accept the task (counted as failed).

4. Implementation

The simulator is written in Python 3 using dataclasses, pandas and matplotlib. It has a command-line menu.

File	Purpose
models.py	Node, Task and VM classes; workload generator; SLA limits
placement.py	Policy A (latency_first) and Policy B (capacity_aware)
security.py	Demo users, tenant/role check, audit log
simulator.py	VM create/stop, task submission, QoS results and summaries
experiments.py	The four required experiments, CSV output and charts
main.py	Command-line menu for the live demonstration

4.1 Virtualization (simulated VMs)

Each placed task creates a VM record containing the VM ID, host node, owning tenant, allocated CPU and RAM, and status (running or stopped). Creating a VM reserves CPU and memory on the host node. Stopping a VM sets its status to stopped and releases those resources. No hypervisor is used; the records are Python objects.

4.2 Basic security and multi-tenancy

Every task record and VM carries a tenant ID. When a user asks to view a task or stop a VM, the simulator checks that the user's tenant matches the owner's tenant, or that the user is an admin. Otherwise an AccessDenied error is raised. Every attempt, allowed or denied, is written to an audit log. The users are fixed demo accounts with no passwords or sensitive data.

4.3 Workload generation

The default workload has 12 tasks with tenants assigned round-robin (tenantA, tenantB, tenantC) and a fixed random seed (42). CPU needs are 1–3 vCPU, memory 1–4 GB, and sensitivity is random with a bias toward high. Tasks can also be added manually from the menu.

4.4 Menu features

The menu supports: showing nodes and tasks, running the workload with either policy, viewing results and the QoS summary, logging in as a user, viewing a task record (access-controlled), listing VMs, creating and stopping VMs, adding a custom task, saving CSV and chart output, running all four experiments, and showing the audit log.

5. Experiments

For each experiment the setup, expected outcome, actual outcome and evidence are given.

5.1 Experiment 1 – Placement policy comparison

Setup. The same 12-task workload (seed 42) is run on fresh default nodes with Policy A and then Policy B.

Expected outcome. Policy B should avoid SLA violations because it only places a task where its latency limit can be met. Policy A should fill Edge and Fog and place some high-sensitivity tasks too far away.

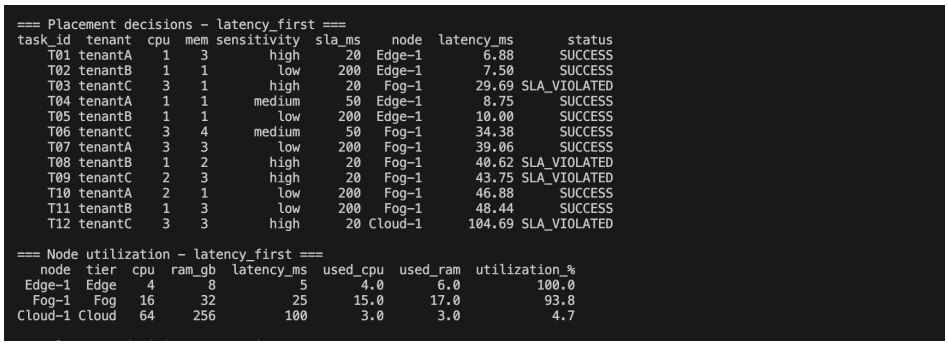
Actual outcome.

Metric	Policy A (latency_first)	Policy B (capacity_aware)
Total tasks	12	12
Successful	8	9
SLA violated	4	0
Rejected	0	3
Success rate	66.7%	75.0%
Average latency (placed tasks)	35.05 ms	67.85 ms
Edge utilization	100%	100%
Fog utilization	93.8%	25.0%
Cloud utilization	4.7%	12.5%

Policy A filled Edge with five tasks (including low and medium ones), then placed high-sensitivity tasks T03, T08 and T09 on Fog (29.69, 40.62 and 43.75 ms) and T12 on Cloud (104.69 ms), all violating the 20 ms limit. Policy B reserved Edge for high-sensitivity tasks T01 and T03, sent medium tasks T04 and T06 to Fog, sent the five low-sensitivity tasks to Cloud, and rejected high-sensitivity tasks T08, T09 and T12 because Edge was full and no other node could meet 20 ms.

Policy B has a higher average latency because it deliberately moves tolerant tasks to Cloud, but it never breaks an SLA and it balances load. Policy A has a lower average latency but 4 SLA violations.

Evidence.



```
=== Placement decisions — latency_first ===
task_id  tenant  cpu  mem sensitivity  sla_ms    node  latency_ms       status
    T01 tenantA    1    3        high      20  Edge-1        6.88      SUCCESS
    T02 tenantB    1    1         low     200  Edge-1        7.50      SUCCESS
    T03 tenantC    3    1        high      20   Fog-1       29.69 SLA_VIOLATED
    T04 tenantA    1    1      medium      50  Edge-1        8.75      SUCCESS
    T05 tenantB    1    1         low     200  Edge-1       10.00      SUCCESS
    T06 tenantC    3    4      medium      50   Fog-1       34.38      SUCCESS
    T07 tenantA    3    3         low     200   Fog-1       39.06      SUCCESS
    T08 tenantB    1    2        high      20   Fog-1       40.62 SLA_VIOLATED
    T09 tenantC    2    3        high      20   Fog-1       43.75 SLA_VIOLATED
    T10 tenantA    2    1         low     200   Fog-1       46.88      SUCCESS
    T11 tenantB    1    3         low     200   Fog-1       48.44      SUCCESS
    T12 tenantC    3    3        high      20 Cloud-1      104.69 SLA_VIOLATED

=== Node utilization — latency_first ===
   node  tier  cpu  ram_gb  latency_ms  used_cpu  used_ram  utilization_%
 Edge-1  Edge    4       8           5       4.0       6.0          100.0
  Fog-1   Fog   16      32          25      15.0      17.0           93.8
Cloud-1 Cloud   64     256         100       3.0       3.0            4.7
```

Figure 2: Experiment 1 – Policy A placement decisions and node utilization.

```
=== Placement decisions — capacity_aware ===
task_id  tenant  cpu  mem sensitivity  sla_ms     node  latency_ms   status
    T01 tenantA    1    3        high      20  Edge-1        6.88  SUCCESS
    T02 tenantB    1    1         low     200 Cloud-1      101.56  SUCCESS
    T03 tenantC    3    1        high      20  Edge-1       10.00  SUCCESS
    T04 tenantA    1    1      medium      50   Fog-1       26.56  SUCCESS
    T05 tenantB    1    1         low     200 Cloud-1      103.12  SUCCESS
    T06 tenantC    3    4      medium      50   Fog-1       31.25  SUCCESS
    T07 tenantA    3    3         low     200 Cloud-1      107.81  SUCCESS
    T08 tenantB    1    2        high      20       -         NaN REJECTED
    T09 tenantC    2    3        high      20       -         NaN REJECTED
    T10 tenantA    2    1         low     200 Cloud-1      110.94  SUCCESS
    T11 tenantB    1    3         low     200 Cloud-1      112.50  SUCCESS
    T12 tenantC    3    3        high      20       -         NaN REJECTED

=== Node utilization — capacity_aware ===
   node  tier  cpu  ram_gb  latency_ms  used_cpu  used_ram  utilization_%
 Edge-1  Edge    4       8           5       4.0       4.0          100.0
  Fog-1   Fog   16      32          25       4.0       5.0           25.0
Cloud-1 Cloud   64     256         100       8.0       9.0           12.5

=== Experiment 1 summary ===
         label  total_tasks  successful  sla_violated  rejected  avg_latency_ms  failed  success_rate_%  util_Edge-1_%  util_Fog-1_%  util_Cloud-1_%
 latency_first           12           8             4         0           35.05       4            66.7          100.0          93.8             4.7
capacity_aware           12           9             0         3           67.85       3            75.0          100.0          25.0            12.5

##### EXPERIMENT 2: Overload scenario #####
Setup: 25 heavy tasks (2–6 CPU, 3–10 GB) sent to default nodes.
Expected: Edge fills first; tasks spill to Fog/Cloud or are rejected when no suitable node remains.

=== Tasks per node — latency_first ===
   node  tasks
      -      5
Cloud-1     14
 Edge-1      2
  Fog-1      4

=== Rejected tasks — latency_first ===
task_id  cpu  mem sensitivity
    T19    6   10      medium
    T22    3    8      medium
    T23    5    9         low
    T24    2    8         low
    T25    4   10      medium

=== Node utilization — latency_first ===
   node  tier  cpu  ram_gb  latency_ms  used_cpu  used_ram  utilization_%
 Edge-1  Edge    4       8           5       4.0       8.0          100.0
  Fog-1   Fog   16      32          25      16.0      28.0          100.0
Cloud-1 Cloud   64     256         100      64.0      80.0          100.0
```

Figure 3: Experiment 1 – Policy B placement decisions, utilization and summary.

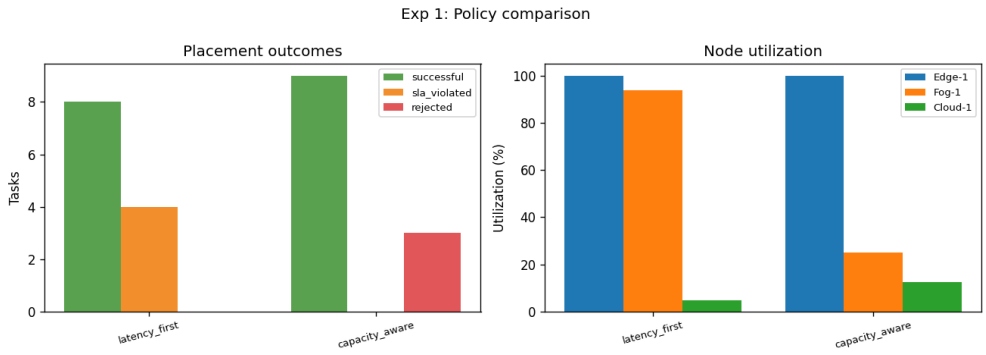


Figure 4: Experiment 1 chart – outcomes and node utilization per policy.

5.2 Experiment 2 – Overload scenario

Setup. A heavy workload of 25 tasks (2–6 vCPU, 3–10 GB, seed 7) is sent to the default nodes, far more than Edge can hold.

Expected outcome. Edge fills first. Further tasks spill to Fog and Cloud, and tasks that no suitable node can hold are rejected.

Actual outcome.

Metric	Policy A (latency_first)	Policy B (capacity_aware)
Successful	9	13
SLA violated	11	0
Rejected	5	12
Success rate	36.0%	52.0%
Average latency	115.58 ms	81.75 ms
Edge / Fog / Cloud utilization	100% / 100% / 100%	100% / 100% / 45.3%
Tasks placed on Edge / Fog / Cloud	2 / 4 / 14	2 / 4 / 7

Under Policy A, tasks spilled across all three nodes until every node was full (100% utilization), five tasks (T19, T22, T23, T24, T25) were rejected, and eleven placed tasks broke their SLA. Under Policy B, tasks were only assigned to nodes that could meet their SLA, so there were no SLA violations, but twelve tasks were rejected (T07, T09, T13, T14, T15, T16, T17, T18, T19, T20, T22, T25), mostly high- and medium-sensitivity tasks that no

node could serve in time. Cloud still had spare capacity (45.3%) under Policy B, but it was too slow for these tasks.

Evidence.

```
=== Tasks per node – capacity_aware ===
   node  tasks
      –     12
Cloud-1      7
 Edge-1      2
  Fog-1      4

=== Rejected tasks – capacity_aware ===
task_id  cpu  mem sensitivity
    T07    6    3        high
    T09    2    5        high
    T13    6    4      medium
    T14    6    6      medium
    T15    6    9      medium
    T16    5   10        high
    T17    3    5      medium
    T18    3    4        high
    T19    6   10      medium
    T20    5    7        high
    T22    3    8      medium
    T25    4   10      medium

=== Node utilization – capacity_aware ===
   node  tier  cpu  ram_gb  latency_ms  used_cpu  used_ram  utilization_%
 Edge-1  Edge    4       8           5       4.0       8.0          100.0
  Fog-1   Fog   16      32          25      16.0      23.0          100.0
Cloud-1 Cloud   64     256         100      29.0      49.0           45.3

=== Experiment 2 summary ===
          label  total_tasks  successful  sla_violated  rejected  avg_latency_ms  failed  success_rate_%  util_Edge-1_%  util_Fog-1_%  util_Cloud-1_%
 latency_first           25           9            11         5          115.58      16            36.0          100.0         100.0           100.0
capacity_aware           25          13             0        12           81.75      12            52.0          100.0         100.0            45.3

##### EXPERIMENT 3: Unauthorized tenant access #####
Setup: 12 tasks placed; alice (tenantA) tries to read tenantB's task and stop tenantB's VM.
Expected: DENIED for cross-tenant; ALLOWED for own tenant and admin.

=== Experiment 3 evidence ===
 user     action      target                                                result
alice view_task   T02 (tenantB) DENIED (alice (tenantA) cannot view T02 owned by tenantB)
alice view_task   T01 (tenantA)                                          ALLOWED
alice   stop_vm VM002 (tenantB)       DENIED (alice cannot stop VM002 (owned by tenantB))
  bob view_task   T02 (tenantB)                                          ALLOWED
admin view_task   T02 (tenantB)                                          ALLOWED

##### EXPERIMENT 4: Before/after node capacity & latency change #####
Setup: capacity_aware policy, same workload (20 tasks, seed 3). Baseline vs changed nodes.
Expected: more Edge capacity -> more high-sensitivity tasks succeed; higher Fog latency -> medium tasks lose Fog and may be rejected.
```

Figure 5: Experiment 2 console output – tasks per node, rejected tasks, utilization and summary.

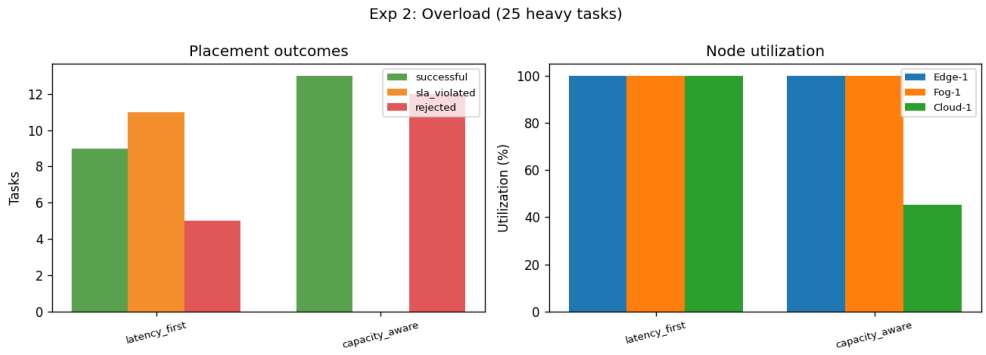


Figure 6: Experiment 2 chart – overload outcomes and node utilization.

5.3 Experiment 3 – Unauthorized tenant access

Setup. After the 12-task workload is placed with Policy B, alice (tenantA) tries to view tenantB's task T02 and to stop tenantB's VM (VM002). Own-tenant and admin access are also tested.

Expected outcome. Cross-tenant access is denied. Access to a user's own tenant records and admin access are allowed.

Actual outcome.

User	Action	Target	Result
alice	view_task	T02 (tenantB)	DENIED
alice	view_task	T01 (tenantA)	ALLOWED
alice	stop_vm	VM002 (tenantB)	DENIED
bob	view_task	T02 (tenantB)	ALLOWED
admin	view_task	T02 (tenantB)	ALLOWED

The same denial was demonstrated live from the menu: alice receives the message that she cannot view T02 owned by tenantB. All attempts are recorded in the audit log.

Evidence.

```
[user: alice | policy: capacity_aware | tasks: 13]
Choose: 6
Task ID (e.g. T01): T02
ACCESS DENIED: alice (tenantA) cannot view T02 owned by tenantB
```

Figure 7: Access denied from the menu – alice viewing tenantB's task T02.

(The Experiment 3 evidence table is also visible in Figure 5.)

5.4 Experiment 4 – Before and after changing node capacity or latency

Setup. A 20-task workload (seed 3) is run with Policy B on the baseline nodes and on three modified configurations: Edge capacity doubled (8 CPU, 16 GB), Fog latency raised from 25 ms to 60 ms, and Cloud latency raised from 100 ms to 150 ms.

Expected outcome. More Edge capacity should let more high-sensitivity tasks succeed. Higher Fog latency should make Fog unusable for medium tasks, increasing rejections.

Actual outcome.

Scenario	Successful	SLA violated	Rejected	Success rate	Avg latency	Edge / Fog / Cloud util
Baseline	15	0	5	75.0%	51.98 ms	100 / 100 / 15.6 %
Edge capacity ×2	16	0	4	80.0%	46.90 ms	100 / 93.8 / 15.6 %
Fog latency 25→60 ms	6	0	14	30.0%	76.56 ms	100 / 0 / 15.6 %
Cloud latency 100→150 ms	15	0	5	75.0%	66.67 ms	100 / 100 / 15.6 %

Doubling Edge capacity raised the success rate from 75% to 80% and lowered the average latency. Raising Fog latency to 60 ms was the most damaging change: Fog could no longer meet the 50 ms limit of medium tasks, so Fog utilization fell to 0% and rejections rose from 5 to 14. Raising Cloud latency only increased the average latency, because the tasks sent to Cloud are low-sensitivity and still within their 200 ms limit.

Evidence.

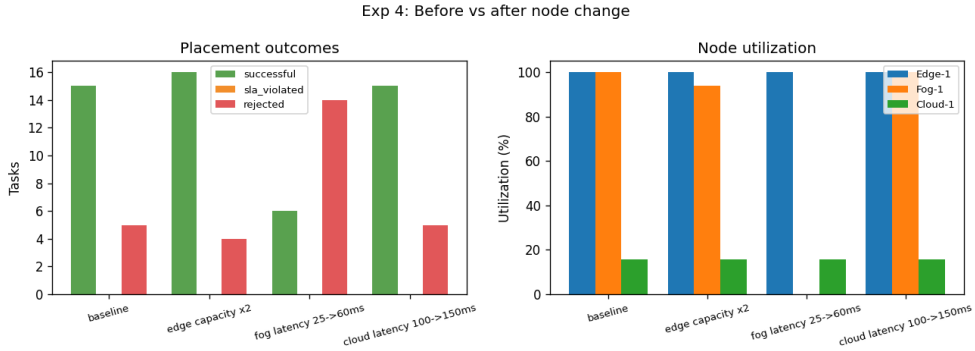


Figure 8: Experiment 4 chart – before and after node changes.

5.5 Virtual machine demonstration

From the menu, the list of VMs is shown (VM ID, host node, tenant, CPU, RAM, status). Stopping a VM changes its status to stopped and releases its CPU and memory on the host node. A user from another tenant cannot stop it.

[Insert Figure 9: screenshot of menu options 7, 9 and 7 – VM list, stop VM001, list again]

6. Results Summary

Experiment	Key result

Experiment	Key result
1. Policy comparison	Policy B: 0 SLA violations, 75.0% success; Policy A: 4 SLA violations, 66.7% success
2. Overload	Policy B: 52.0% success, 0 SLA violations; Policy A: 36.0% success, 11 SLA violations
3. Unauthorized access	Cross-tenant view and VM stop denied; own-tenant and admin access allowed
4. Before/after change	Edge ×2: 75% to 80%; Fog latency 60 ms: 75% to 30%; Cloud latency 150 ms: 75% unchanged

Across the experiments, the capacity-aware policy trades a higher average latency and more rejections for strict SLA compliance and better load balancing. The latency-first policy gives low latency on average but overloads Edge and Fog and then breaks SLAs under load.

7. Limitations

- Latency is a simple formula (base latency × (1 + utilization)); there is no real network, bandwidth, jitter or queueing.
- Tasks have no duration: they hold CPU and memory until their VM is stopped, so there is no time-based simulation or automatic completion.
- There is one node per tier. Multiple Edge, Fog or Cloud nodes, migration and failure handling are not modeled.
- The placement policies are rule-based and greedy; they do not optimize globally or learn from history, and rejected tasks are not retried or queued.
- Security is a demo tenant/role check with fixed users and no real authentication, encryption or network isolation.
- Cost and energy use are not modeled.
- Results depend on the fixed random seeds used for the workloads.

8. Conclusion

The simulator meets the project requirements: it models Edge, Fog and Cloud nodes, generates workloads, places tasks with two rule-based policies, keeps simulated VM records, enforces tenant isolation and reports QoS results. The experiments show that a latency-first rule is simple but breaks SLAs under load, while a capacity-aware rule that respects each task's latency limit avoids SLA violations at the price of more rejections and higher average latency. Node capacity and latency strongly affect outcomes, with Fog latency having the largest effect in these tests.